

Politechnika Warszawska

Wydział Matematyki
i Nauk Informacyjnych

Edytor schematów szydełkowych z automatycznym rozmieszczaniem symboli

Projekt architektury i projekt szczegółowy systemu

Autorzy:

Anna Babicka, Wiktor Murawski

Promotor:

dr inż. Paweł Kotowski

31 października 2026

Spis treści

1 Architektura systemu **4**
1.1 Przegląd logiczny 4
1.2 Odpowiedzialności modułów 5

2 Projekt modułów **6**
2.1 Warstwy i zależności 6
2.2 Moduły szczegółowe (katalogi) 7

3 Komunikacja **9**

4 Projekt szczegółowy: model danych i zachowanie systemu **10**
4.1 Model danych (Domena) 10
4.2 Stan układu symboli 10
4.3 Aktywność: dodanie oczka (US-02) 11
4.4 Zdarzenia kluczowe 11

5 Wizja interfejsu użytkownika **12**
5.1 Główne obszary (wireframe tekstowy) 12
5.2 Kluczowe interakcje 12

6 Zewnętrzne interfejsy **13**
6.1 Format pliku projektu (JSON) 13
6.2 Eksport graficzny 13

7 Stos technologiczny **14**
7.1 Uzasadnienie wyboru podejścia do layoutu 15
7.2 Środowisko deweloperskie 15

Bibliografia **16**

Historia zmian dokumentu

Wersja	Data	Autor	Opis zmian
1.0	29.10.2026	Wiktor Murawski	*Pierwsza wersja dokumentu – szkielet architektury, wybór technologii, opis modułów*

1 Architektura systemu

1.1 Przegląd logiczny

Rysunek 1 przedstawia ogólną, logiczną architekturę systemu: główne moduły aplikacji i zależności między nimi, bez odwołań do konkretnych technologii. Strzałki wskazują kierunek wywołań między modułami; moduł wywoływany zwraca wynik do modułu wywołującego. Linia przerywana oznacza dostęp tylko do odczytu.

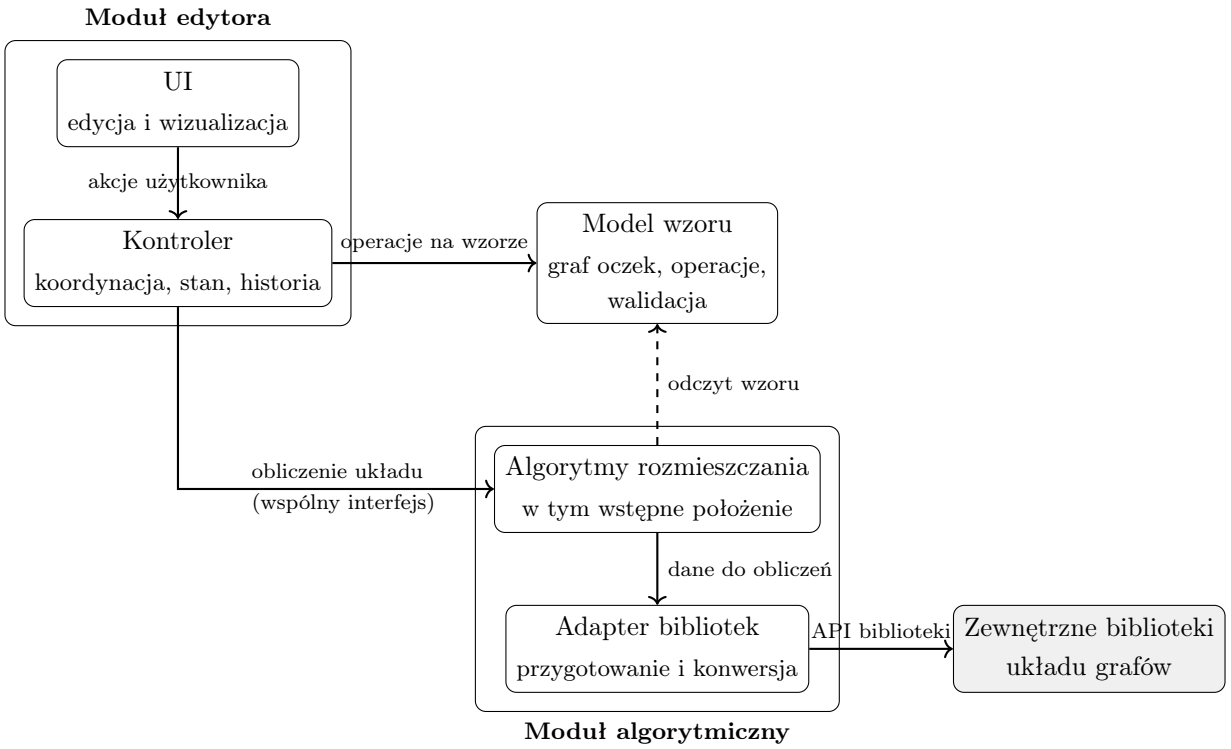
System składa się z trzech głównych części.

Moduł edytora odpowiada za interakcję z użytkownikiem. Warstwa UI umożliwia edycję i wyświetla schemat, a kontroler koordynuje przepływ informacji między interfejsem a pozostałymi modułami. Kontroler przechowuje także bieżący stan aplikacji i historię operacji (wzoru i układu symboli), dzięki czemu warstwa UI jest odizolowana od logiki dziedzinowej.

Model wzoru przechowuje formalną reprezentację wzoru w postaci grafu oczek wraz z operacjami jego modyfikacji i regułami zapewniającymi, że wzór jest możliwy do wykonania. Stanowi jedyne źródło danych o strukturze wzoru. Zmiany wprowadza wyłącznie kontroler, a moduł algorytmiczny jedynie odczytuje strukturę wzoru.

Moduł algorytmiczny na podstawie struktury wzoru wyznacza układ symboli na schemacie, w tym wstępne położenie nowo dodanych elementów. Komunikuje się z kontrolerem przez wspólny interfejs, co pozwala rozwijać edytor i algorytm niezależnie. W razie potrzeby korzysta z zewnętrznych bibliotek układu grafów. Komunikację z nimi obsługuje adapter, który przygotowuje dane w wymaganym formacie i przekształca wyniki. Dzięki temu zmiana wykorzystywanej biblioteki nie wpływa na pozostałe elementy systemu.

Taki podział oddziela strukturę wzoru od jego reprezentacji graficznej: model opisuje, *co* zostało zaprojektowane, moduł algorytmiczny określa, *jak* przedstawić to na schemacie, a edytor pośredniczy między użytkownikiem a tymi dwoma modułami.



Rysunek 1: Logiczna architektura systemu – główne moduły i zależności

1.2 Odpowiedzialności modułów

Moduł	**Główne odpowiedzialności**
UI	Renderowanie symboli schematu, obsługa gestów (kliknięcie, przeciąganie, przybliżanie, przesuwanie widoku niezależnie od układu), paleta symboli, panele właściwości, komunikaty błędów, wskaźniki postępu obliczeń layoutu.
Kontroler	Koordynacja przepływu; przechowywanie bieżącego stanu aplikacji i historii operacji (cofnij/ponów); wywoływanie operacji na modelu oraz żądań obliczenia układu; zapis i odczyt projektu (wzór i układ), eksport schematu do plików graficznych; izolacja UI od logiki dziedzinowej.
Model wzoru	Jedne źródło danych o strukturze wzoru (graf oczek); operacje atomowe (dodanie/usunięcie/zmiana typu oczka, nowa warstwa); reguły walidacji poprawności wzoru; brak zależności od UI i algorytmu.
Algorytmy rozmieszczania	Wyznaczanie pozycji, orientacji i skali symboli; wstępne położenie nowo dodanych elementów; metryki jakości układu (nakładanie, proporcje).
Adapter bibliotek	Przygotowanie danych wejściowych dla zewnętrznej biblioteki układu grafów; konwersja wyników do wewnętrznego formatu modułu algorytmicznego; izolacja reszty systemu od konkretnej implementacji.

2 Projekt modułów

2.1 Warstwy i zależności

Architektura jest warstwowa z wyraźnym kierunkiem zależności (warstwy wyższe zależą od niższych, nigdy odwrotnie):

1. **Prezentacja (UI)** – komponenty React, hooki; zależy wyłącznie od kontrolera.
2. **Aplikacja (Kontroler)** – stan, historia, koordynacja; zależy od modelu wzoru i kontraktu modułu algorytmicznego.
3. **Domena (Model wzoru)** – model wzoru oraz algorytmy rozmieszczania; czysty TypeScript, bez Reacta i DOM; niezależny.
4. **Infrastruktura** – Web Worker, adapter bibliotek, localStorage, File API; implementuje interfejsy zdefiniowane wyżej (odwrócenie zależności).

Struktura modelu wzoru opiera się na grafowej reprezentacji wzorów szydełkowych zaproponowanej przez Seitz i in. [1, 2]; jej założenia i wprowadzone zmiany opisano w punkcie 4.1.

Kluczowe interfejsy (szkic TypeScript):

```
// Domain
interface Stitch {
  id: string;
  type: StitchType;           // sc | hdc | dc | tr | ch | slst | mr ...
  layer: number;
  // ... dodatkowe atrybuty (np. yarn-over count)
}

interface Edge {              // kierunek: od nowszego do starszego węzła
  kind: EdgeKind;             // 'previous' | 'insertion' | 'slip-stitch' | 'surrounding'
  from: StitchId;
  to: StitchId;
  insertion?: Insertion;      // tylko dla kind = 'insertion'; np. 'both-loops' | 'back-loop' ..
}

interface Pattern  {
  stitches: Record<string, Stitch>;
  edges: readonly Edge[];
  cursor: StitchId | null;    // oczko, z którego wychodzi nić
  currentLayer: number;
  nextOrder: number;          // licznik do nadawania id i kolejności
}

type Result<T> = { ok: true; value: T } | { ok: false; error: string };
```

```
interface DomainAPI {
  createNewPattern(start: StartElement): Pattern;
  addStitch(pattern: Pattern, type: StitchType,
      insertInto?: StitchId | StitchId[],
      insertion?: Insertion): Result<Pattern>;
  removeLastStitch(pattern: Pattern): Result<Pattern>;
  // ... pozostałe operacje z US-01..US-05
  validateInsertion(type: StitchType, target: StitchId): boolean;
}

// Layout
interface StitchPlacement {
  x: number;           // środek symbolu, jednostki modelu
  y: number;
  rotation: number;    // radiany; 0 = podstawa symbolu na dole
  sw: number;          // skala szerokości: W = baseWidth * sw
  sh: number;          // skala wysokości:  H = baseHeight * sh
}

interface Layout {
  placements: Readonly<Record<StitchId, StitchPlacement>>;
  bounds: { minX: number; minY: number; maxX: number; maxY: number };
}

type LayoutResult =
  | { ok: true; layout: Layout; diagnostics: LayoutDiagnostics }
  | { ok: false; error: string };

interface LayoutEngine {
  computeLayout(p: Pattern, options?: LayoutOptions): Promise<LayoutResult>;
}
```

2.2 Moduły szczegółowe (katalogi)

Kod źródłowy jest podzielony na katalogi odpowiadające modułom z rysunku 1:

- **contracts**/ – typy i interfejsy współdzielone przez moduły, w szczególności kontrakt modułu algorytmicznego. Nie zawiera logiki.
- **model**/ – model wzoru: graf oczek, operacje jego modyfikacji, reguły walidacji oraz katalog rodzajów oczek z ich geometrią.
- **layout**/ – moduł algorytmiczny: wstępne rozmieszczenie nowych oczek, wymienne algorytmy pełnego przeliczenia układu i adaptery zewnętrznych bibliotek.

- **controller**/ – stan aplikacji (wzór i układ), historia operacji, koordynacja wywołań.
- **ui**/ – interfejs użytkownika: obszar schematu, panele i symbole oczek.
- **persist**/ – zapis i odczyt projektu oraz eksport schematu.

Katalogi `model/` i `layout/` nie zależą od interfejsu użytkownika ani od API przeglądarki, dzięki czemu można je testować niezależnie (por. NFR-M1, NFR-M3), a wymiana biblioteki układu grafów wymaga zmian wyłącznie w module algorytmicznym.

3 Komunikacja

Aplikacja działa w całości po stronie klienta, dlatego nie występują w niej protokoły sieciowe ani komunikacja z serwerem. Komunikacja między modułami przebiega następująco:

- **UI ↔ Kontroler** – UI przekazuje akcje użytkownika do kontrolera, a komponenty subskrybują wybrane fragmenty stanu. Zmiana stanu powoduje ponowne wyrenderowanie wyłącznie komponentów, których dotyczy.
- **Kontroler ↔ Model wzoru** – synchroniczne wywołania czystych funkcji. Każda operacja zwraca nową wersję wzoru albo opis błędu; nowa wersja wraz z układem symboli trafia do historii operacji.
- **Kontroler ↔ Moduł algorytmiczny** – dwa tryby:
  - *wstępne rozmieszczenie* nowo dodanych oczek – synchroniczne wywołanie w głównym wątku po każdej zmianie wzoru, bez zmiany położenia pozostałych symboli,
  - *pełne przeliczenie układu* – na żądanie użytkownika, asynchronicznie w osobnym wątku (Web Worker). Kontroler wysyła komunikat (`postMessage`) z wzorem, poprzednim układem i parametrami obliczeń, a wątek zwraca `LayoutResult`. Każde żądanie ma identyfikator; odpowiedzi na żądania nieaktualne lub anulowane są pomijane. W czasie obliczeń interfejs pozostaje responsywny (NFR-P3).
- **Adapter ↔ biblioteka zewnętrzna** – wywołania API wybranej biblioteki układu grafów wewnątrz wątku obliczeniowego, z konwersją danych w obu kierunkach.
- **Zapis i odczyt** – projekt (wzór i układ) jest zapisywany do pliku pobieranego przez przeglądarkę i wczytywany przez standardowe okno wyboru pliku; w przeglądarkach, które to obsługują, można dodatkowo użyć File System Access API. Autozapis do pamięci lokalnej przeglądarki co najwyżej co 30 s (NFR-R3).

Do komunikacji wykorzystywane są wyłącznie mechanizmy wbudowane w przeglądarkę: API Web Workerów i algorytm structured clone przy przesyłaniu danych między wątkami.

4 Projekt szczegółowy: model danych i zachowanie systemu

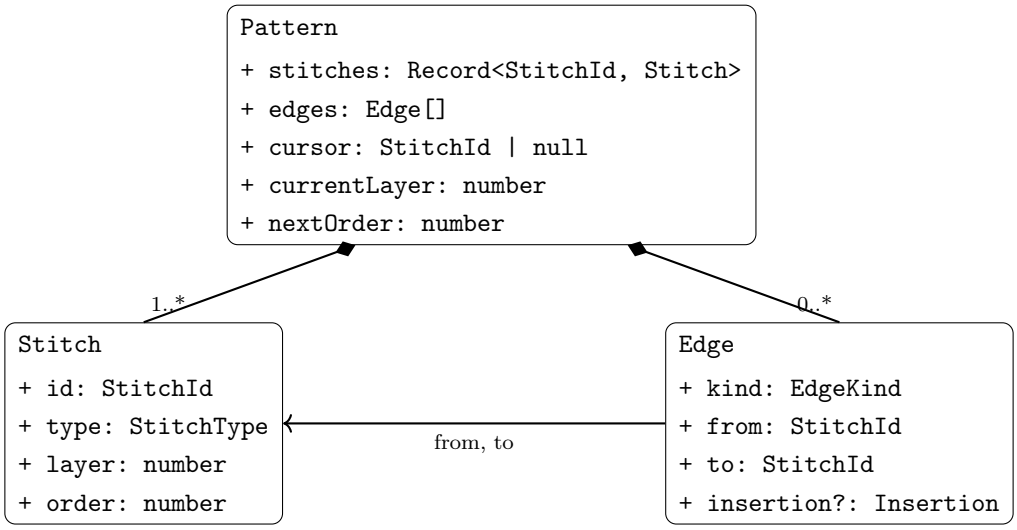
4.1 Model danych (Domena)

Wzór jest reprezentowany jako graf skierowany. Węzły odpowiadają punktom wbicia – najczęściej oczkom, ale także otworom, w które można wbić kolejne oczka. Krawędzie są skierowane od węzła nowszego do starszego i należą do jednego z czterech rodzajów: *previous* (skąd wychodzi nić), *insertion* (w co oczko jest wbite, wraz ze sposobem wbicia), *slip-stitch* (oczko ścisłe) oraz *surrounding* (połączenie otworu z oczkami na jego brzegu). Wzór przechowuje również pozycję nici (kursor) i numer bieżącej warstwy. Układ symboli nie jest częścią modelu – wyznacza go moduł algorytmiczny.

Model danych został oparty na grafowej reprezentacji wzorów szydełkowych zaproponowanej przez Seitz i in. [1, 2]. Przejęto z niej podstawowe założenia: węzły grafu odpowiadają punktom wbicia, a nie bezpośrednio oczkom; krawędzie opisują pochodzenie nici (*previous*), wbicie oczka (*insertion*) i oczko ścisłe (*slip-stitch*); otwory w robótce są reprezentowane jako osobne węzły połączone z oczkami na ich brzegu; rzędy i okrążenia są ujednolicone w pojęcie warstwy, a sposób wbicia (np. tylko pod tylną nitkę) jest właściwością krawędzi wbicia.

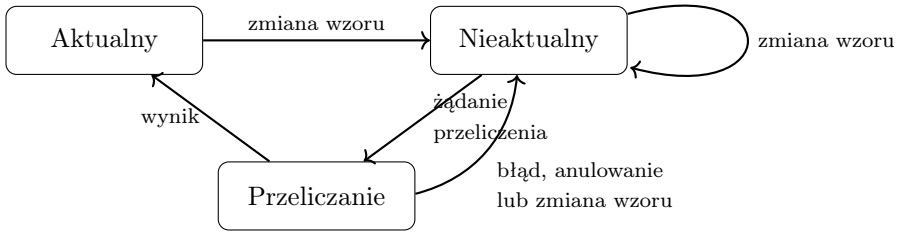
W stosunku do rozwiązania autorów wprowadzono następujące zmiany:

- informacje o połączeniach między oczkami są przechowywane wyłącznie w krawędziach, bez powielania ich w węzłach,
- model nie zawiera informacji o położeniu symboli – układ jest wyznaczany i przechowywany oddzielnie przez moduł algorytmiczny,
- każde oczko ma numer porządkowy, a wzór przechowuje bieżącą pozycję nici, co upraszcza operacje edycji.



4.2 Stan układu symboli

Po każdej zmianie wzoru nowe oczka otrzymują wstępne położenie, a układ zostaje oznaczony jako nieaktualny. Pełne przeliczenie odbywa się na żądanie użytkownika.



4.3 Aktywność: dodanie oczka (US-02)

1. Użytkownik wybiera rodzaj oczka z palety.
2. Użytkownik klika punkt wbicia na schemacie; UI przekazuje akcję do kontrolera.
3. Kontroler wywołuje operację modelu `addStitch`, która sprawdza poprawność wbicia.
4. Jeśli operacja się nie powiedzie, UI wyświetla komunikat z przyczyną, a wzór pozostaje bez zmian.
5. W przeciwnym razie kontroler wywołuje `placeNewStitches`, które wyznacza położenie nowego oczka bez zmiany położenia pozostałych.
6. Kontroler zapisuje nową parę wzór + układ w historii operacji i oznacza układ jako nieaktualny.
7. UI wyświetla nowe oczko na schemacie.

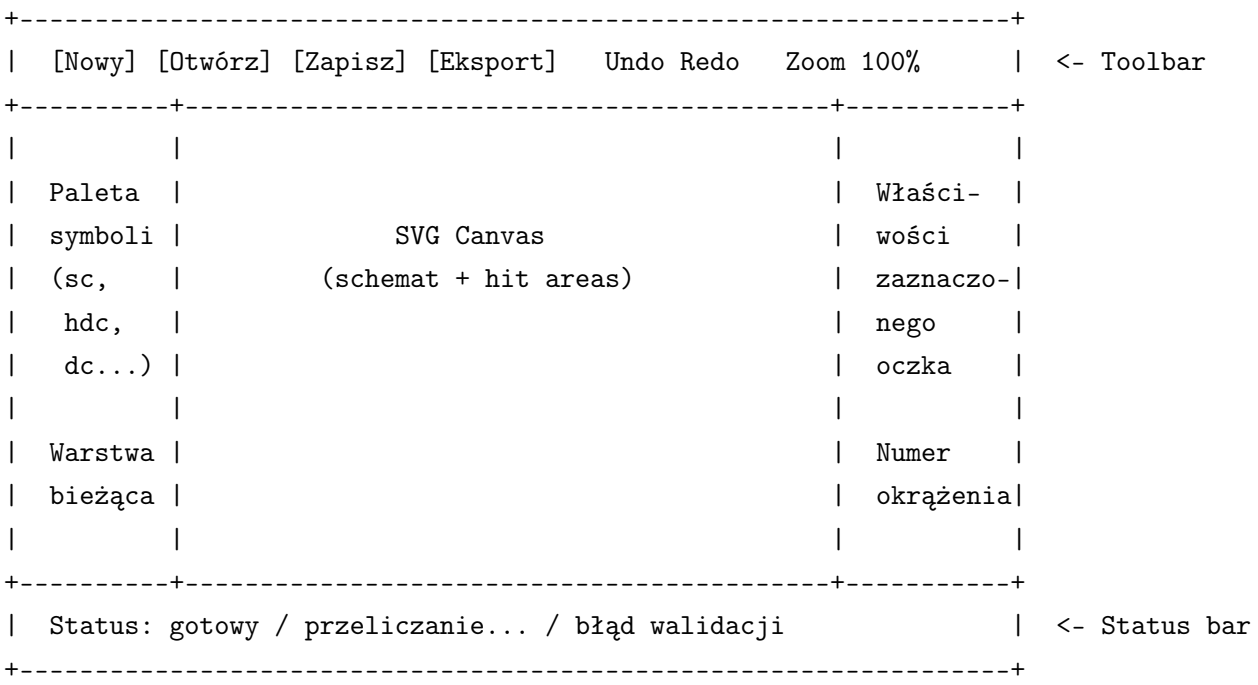
4.4 Zdarzenia kluczowe

- `pattern:changed` – po każdej mutacji grafu.
- `layout:started / layout:finished / layout:failed`.
- `selection:changed`, `view:transformed` (zoom/pan).
- `history:undo / history:redo`.
- `persist:autosaved`, `export:requested`.

5 Wizja interfejsu użytkownika

Interfejs jest jednookienkowy, zorientowany na maksymalną przestrzeń roboczą dla schematu.

5.1 Główne obszary (wireframe tekstowy)



5.2 Kluczowe interakcje

- Kliknięcie symbolu w palecie + kliknięcie na schemacie = dodanie oczka (US-02).
- Kółko myszy / przyciski +/− = zoom względem kursora (US-07).
- Przeciąganie tła = pan.
- Ctrl+Z / Ctrl+Y = undo/redo (US-06).
- Podwójne kliknięcie lub panel właściwości = zmiana typu / parametrów.
- Wskaźnik postępu / spinner na canvasie podczas długiego layoutu (>200 ms).

Wizualizacje (makiety) zostaną doprecyzowane w kolejnej iteracji dokumentu lub w osobnym załączniku (Figma / szkice). Na obecnym etapie wystarcza opis obszarów i mapowanie na historyjki użytkownika.

6 Zewnętrzne interfejsy

6.1 Format pliku projektu (JSON)

Własny, wersjonowany format JSON (schema w osobnym pliku JSON Schema):

```
{
  "version": "1.0",
  "meta": { "title": "...", "created": "...", "seed": 12345 },
  "pattern": {
    "stitches": [ { "id": "...", "type": "dc", "parents": ["..."], "round": 2 }, ... ],
    "yarnPosition": "...",
    "currentRound": 3
  },
  "view": { "zoom": 1.0, "panX": 0, "panY": 0 }   // opcjonalnie
}
```

Zachowanie 100% danych strukturalnych (NFR-R1). Niepoprawny plik → komunikat błędu, brak awarii (NFR-R2).

6.2 Eksport graficzny

- **SVG** – natywny, wektorowy, zachowuje symbole jako `<g>` / `<path>`.
- **PNG / JPEG** – rasteryzacja SVG przez OffscreenCanvas lub tymczasowy `<canvas>`.
- **PDF** – jsPDF + svg2pdf.js lub podobna biblioteka; jedna strona A4 / Letter z dopasowanym schematem.

Brak integracji z zewnętrznymi usługami (brak OAuth, API chmurowych itp.). Standardy: SVG 1.1 / 2, JSON (RFC 8259), File API / File System Access API.

7 Stos technologiczny

Poniższa tabela przedstawia technologie przyjęte w projekcie wraz z rozważanymi alternatywami i uzasadnieniem wyboru. Aplikacja działa w całości w przeglądarce, dlatego narzędzia takie jak Node.js czy Vite są wykorzystywane wyłącznie na etapie tworzenia, testowania i budowania aplikacji.

Moduł / Komponent	**Wybrana technologia**	**Rozważane alternatywy**	**Uzasadnienie wyboru**
Platforma	aplikacja webowa	Aplikacja desktopowa: WPF/C#, Avalonia/C#, Electron	Dostęp bez instalacji, jedna implementacja dla różnych systemów operacyjnych.
Framework	React	Vue	Dojrzały ekosystem, model komponentowy, dobra współpraca z niemutowalnym stanem.
Język	TypeScript	JavaScript	Statyczne typowanie złożonego modelu danych, bezpieczniejsza refaktoryzacja, wspólne kontrakty między modułami.
Renderowanie schematu	SVG	Canvas (Konva / react-konva), czysty HTML Canvas	Grafika wektorowa, każdy symbol jako osobny element, naturalny format eksportu; Canvas wymagałby osobnego eksportu do SVG i własnej obsługi zaznaczania.
Narzędzie budowania	Vite	Webpack	Szybki serwer deweloperski, gotowa obsługa TypeScriptu i Reacta.
Środowisko uruchomieniowe (dev)	Node.js + npm	–	npm dostępny razem z Node.js bez dodatkowej instalacji; wersje przypięte w `.nvmrc` i lockfile'u zapewniają powtarzalność środowiska.
Repozytorium i CI/CD	GitHub + GitHub Actions	GitLab CI	Code review przez pull requesty, automatyczne testy przy każdej zmianie, wcześniejsze doświadczenie z GitHub Actions.
Zarządzanie stanem	Zustand (lub Jotai)	Redux Toolkit, React Context	Lekki store, łatwe middleware undo/redo, mniejszy boilerplate.
Silnik layoutu	d3-force / WebCola	Własna implementacja, cytoscape.js	Dojrzałe, dobrze udokumentowane implementacje force-directed + constraint-based; możliwość uruchomienia w Web Workerze; izolacja przez adapter.

Moduł / Komponent	**Wybrana technologia**	**Rozważane alternatywy**	**Uzasadnienie wyboru**
Testy	Vitest + Testing Library	Jest	Natywna integracja z Vite, szybkie uruchamianie, dobre wsparcie TypeScript.
Eksport PDF	jsPDF + svg2pdf	pdfkit, puppeteer	Działa w pełni po stronie klienta, bez potrzeby serwera.

7.1 Uzasadnienie wyboru podejścia do layoutu

Na podstawie literatury (m.in. Greer & Mould 2025 – *Modeling Crochet Patterns with a Force-directed Graph Layout*) oraz analizy istniejących narzędzi (CrochetPARADE, Stitch Fiddle) wybrano podejście force-directed / constraint-based. Nie tworzymy nowego algorytmu od zera; wartość inżynierska leży w integracji, dostosowaniu sił (orientacja symboli względem rodzica, preferowane proporcje H/W, minimalizacja nakładania) oraz zapewnieniu interaktywności (Web Worker, determinizm przy seedzie). Adapter bibliotek pozwala na ewentualną wymianę implementacji bez wpływu na resztę systemu.

7.2 Środowisko deweloperskie

Node.js + npm/pnpm, ESLint + Prettier, Git. Testy w CI (GitHub Actions). Brak zależności od konkretnego systemu operacyjnego – rozwój możliwy na Windows / macOS / Linux. Wersje narzędzi są przypięte (`.nvmrc`, `package-lock.json` / `pnpm-lock.yaml`).

Bibliografia

[1] Klara Seitz i in. „Digital Crochet: Toward a Visual Language for Pattern Description". W: *Proceedings of the 2022 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! '22)*. ACM, 2022, s. 48–62. DOI: 10.1145/3563835.3567657.

[2] Klara Seitz i in. *Language and Tool Support for 3D Crochet Patterns: Virtual Crochet with a Graph Structure*. Technische Berichte des Hasso-Plattner-Instituts für Digital Engineering 137. Hasso-Plattner-Institut, Universität Potsdam, 2021. DOI: 10.25932/publishup-49253.